

GitHub Engineering Intelligence

Recommended Tech Stack & System Architecture

This document proposes a tech stack sized for a personal / SDE-1 portfolio project, following the three-brain architecture from the original idea: a deterministic analytics + detection engine, and an LLM used only for natural-language explanation and Q&A via MCP.

Recommended Tech Stack

1. Data Ingestion & Sync

GitHub access	GitHub REST API + GraphQL API (for efficient bulk queries), GitHub App or OAuth App for auth
Sync worker	Python + PyGithub (or raw requests/httpx), triggered by webhooks (PR/issue/review events) plus a scheduled poll as backup
Scheduling	APScheduler for a simple personal project; Celery + Redis if you want a more "production-grade" story for interviews

2. Storage

Database	PostgreSQL — relational fits PRs/issues/reviews/commits well and makes the analytics SQL layer easy
Hosting	Neon or Supabase (free tier, serverless Postgres) for a personal project
ORM	SQLAlchemy (with Alembic for migrations) or SQLModel for a lighter-weight typed layer

3. Analytics + Detection + Recommendation Engines

Language	Python
Libraries	pandas / plain SQL aggregations for metrics (cycle time, review wait, merge wait)
Detection logic	Plain Python rule engine (thresholds + z-scores/percentage change) — deliberately no LLM here, per the original design
Scoring	Simple weighted-average health model per dimension, stored per snapshot so you can chart trend over time

4. Backend API

Framework	FastAPI — async, typed, auto-generates OpenAPI docs, integrates cleanly with an MCP server
Auth	GitHub OAuth for login; JWT or session cookies for your own app auth

Background jobs	FastAPI background tasks for light work; Celery worker for heavier/longer sync jobs
------------------------	---

5. AI / LLM Layer

Explanation LLM	Anthropic Claude API (e.g. Claude Sonnet) — receives structured JSON (health score, findings, metrics) and returns natural-language explanation only
Conversational Q&A	Same Claude API, called from an MCP-compatible client/agent so it can call your tools instead of receiving a data dump
MCP server	Anthropic's official Python MCP SDK, exposing tools like <code>get_project_health()</code> , <code>get_open_prs()</code> , <code>get_review_bottlenecks()</code> , etc.

6. Frontend

Framework	React with Next.js (file-based routing, easy deploy to Vercel)
Styling	Tailwind CSS
Charts	Recharts or Chart.js for health score trend, PR cycle time breakdown, reviewer load
Chat UI	A simple chat panel that streams the LLM's answer, calling your MCP-backed API

7. Deployment (personal-project friendly)

Frontend	Vercel (free tier, trivial Next.js deploys)
Backend	Render or Railway (free/cheap tier, easy FastAPI + worker deploys)
Database	Neon or Supabase (managed Postgres, free tier)
Secrets	Environment variables via the host's dashboard; never commit GitHub/Anthropic keys

System Architecture Diagram

The diagram below reflects the three-brain design from the original idea: GitHub data flows down through the sync worker into Postgres, is analyzed and scored deterministically, explained by an LLM, and exposed to users both via a dashboard and via an MCP-backed chat interface.

GitHub Engineering Intelligence — System Architecture

